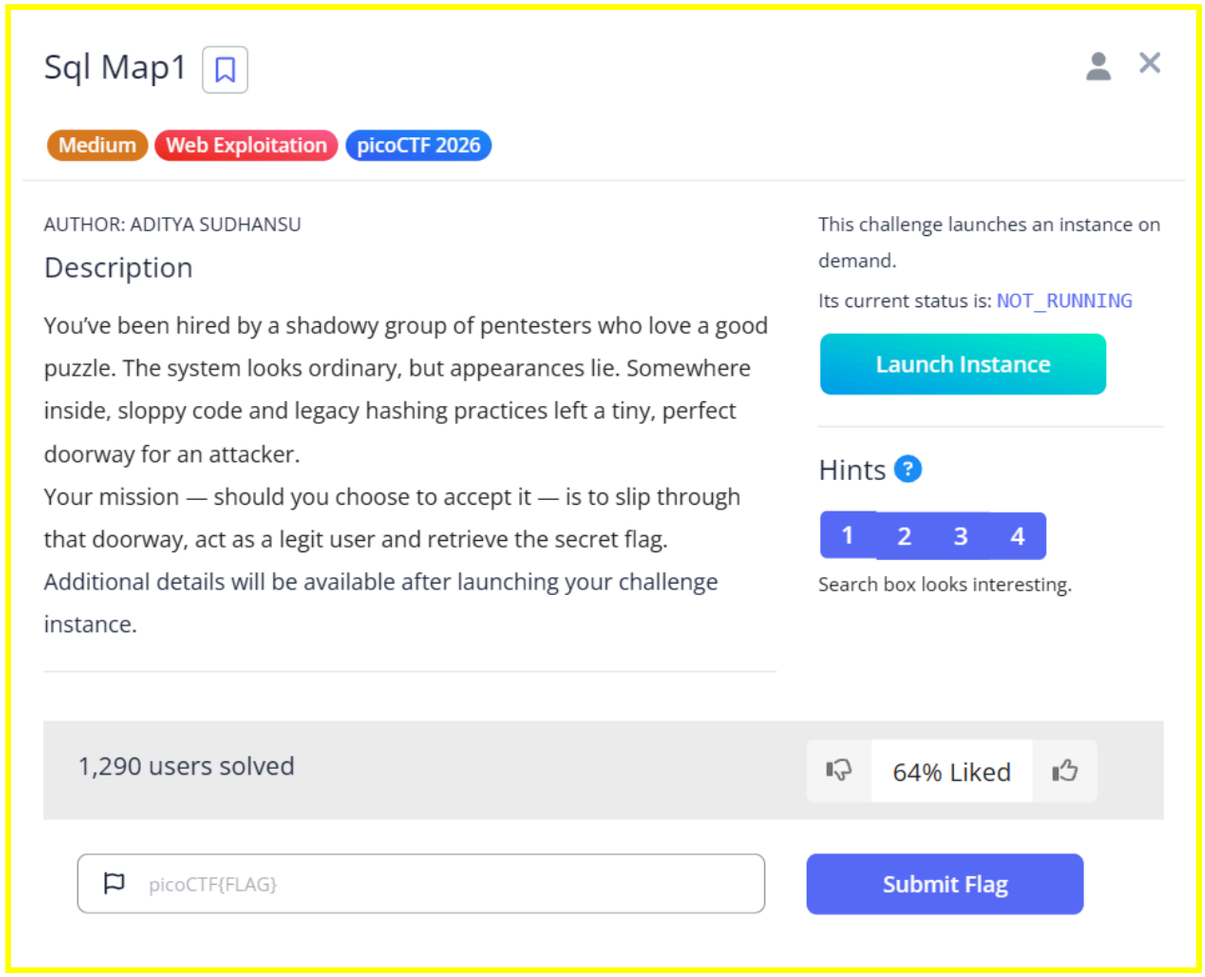
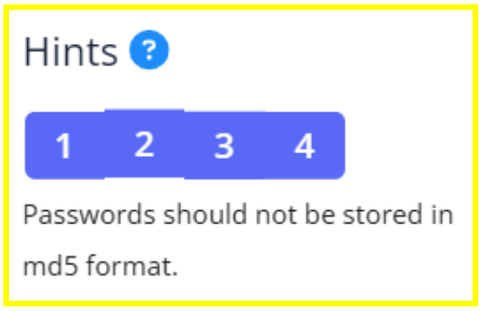
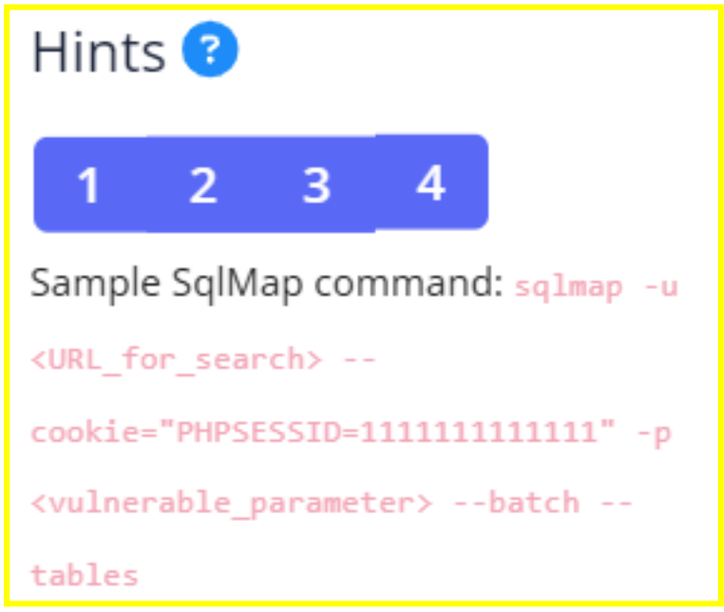
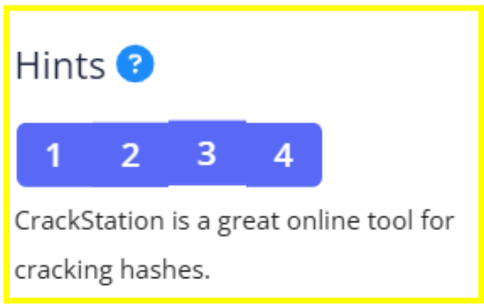
CHALLENGE #5: SQL Map1 Write-up.





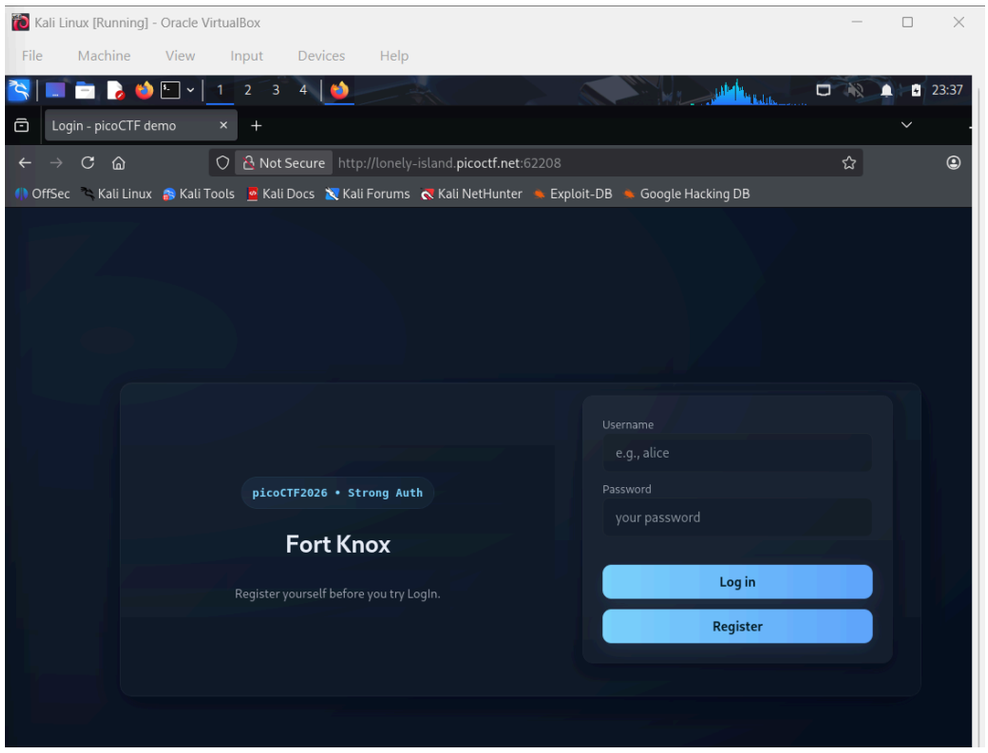


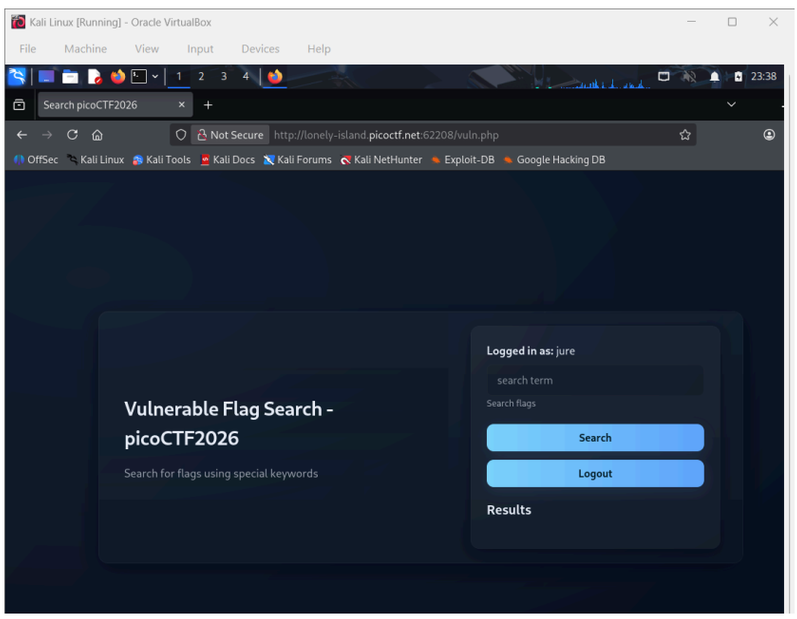


Solving Process:

The challenge began by opening a website that required users to register and log in before accessing the system. The challenge description suggested that although the system appeared ordinary, there might be weaknesses in the code that could allow an attacker to retrieve the

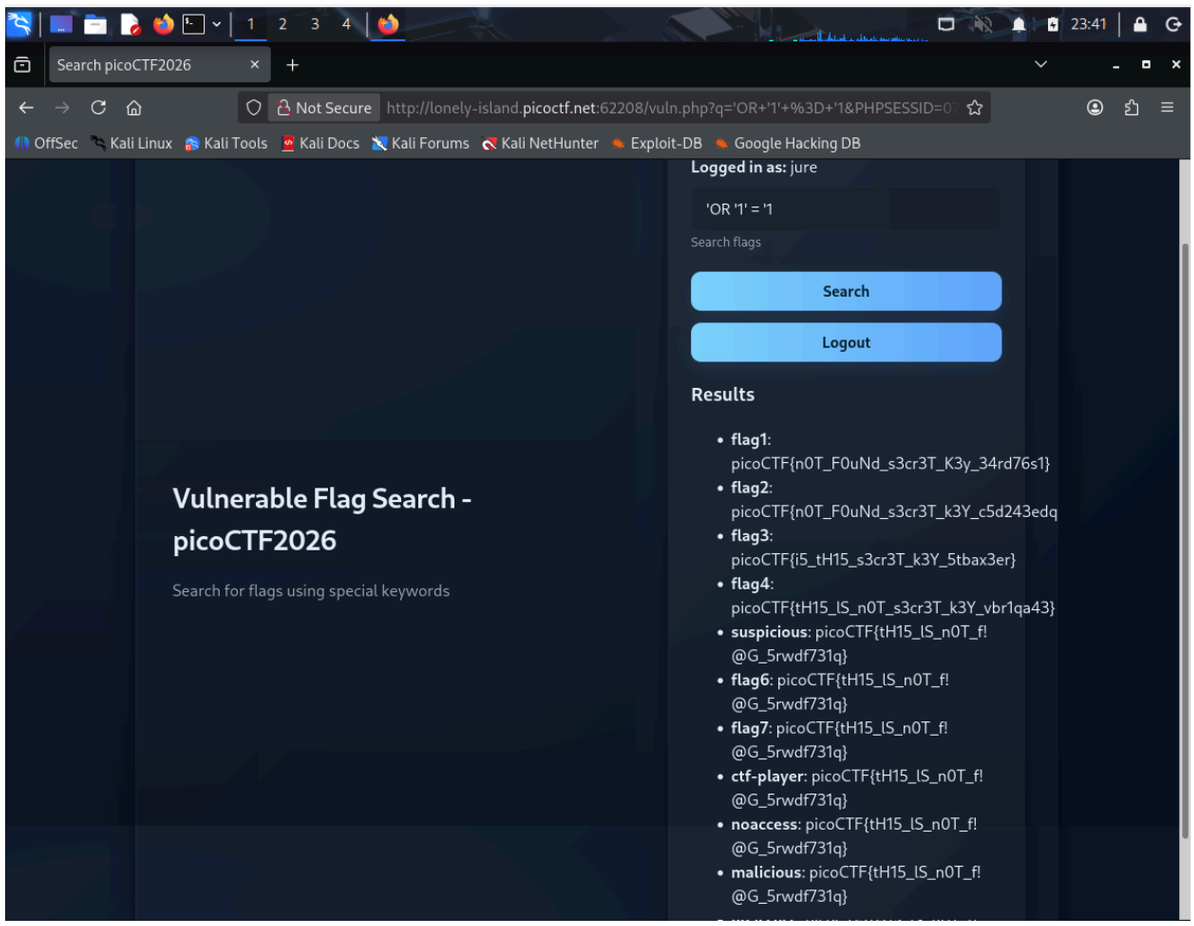
secret flag. The hints also suggested examining the search box, noted that passwords were stored in MD5 format, and recommended using tools such as CrackStation to crack hashes.



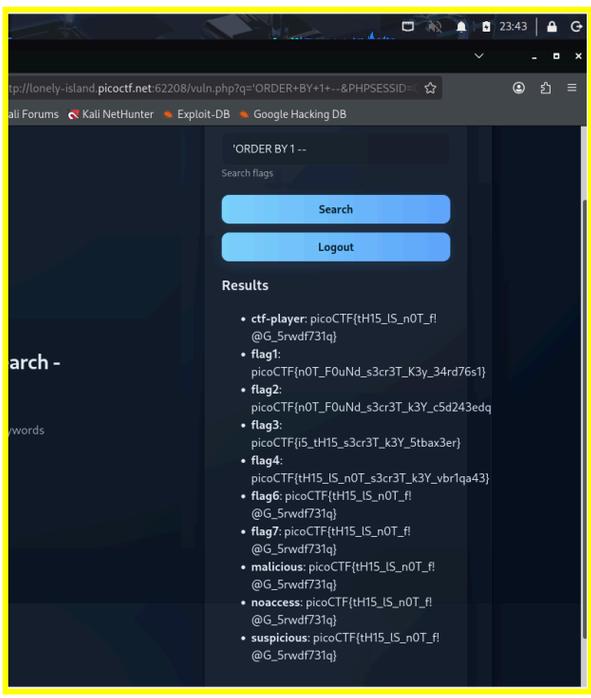


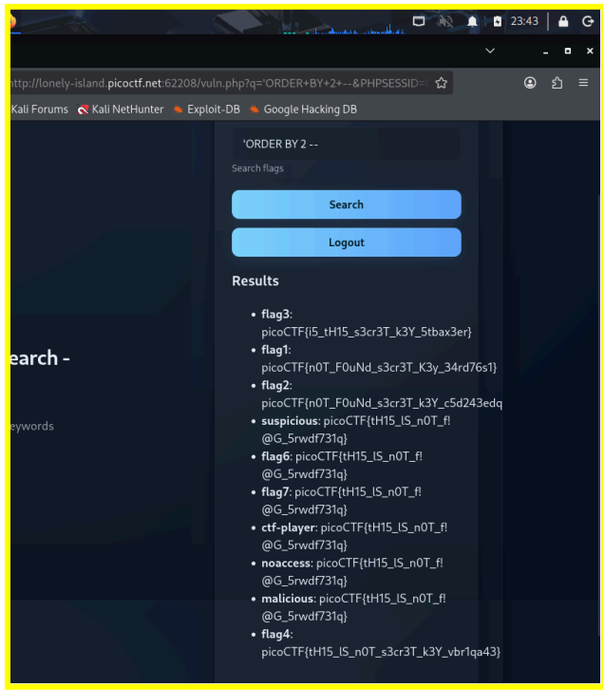
To begin exploring the application, a new account was first created through the registration page. After registering a username and password, the account was used to successfully log in to the system. Once logged in, the page displayed a search function that allowed users to search for flags using keywords. Since the hints specifically mentioned that the search box looked interesting, it was suspected that this input field might be vulnerable to SQL injection.

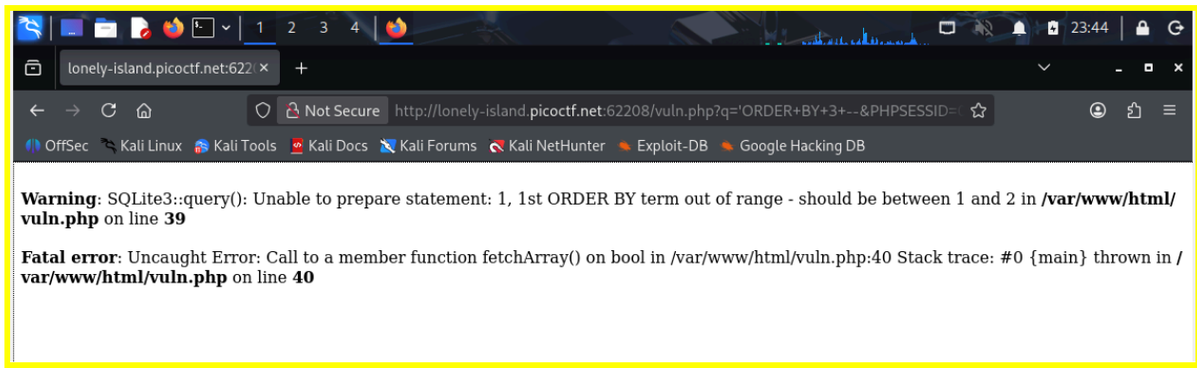
To test this, a simple SQL injection payload was entered into the search field. The results displayed multiple flag-like outputs, but they were clearly dummy flags, meaning they were intentionally placed to mislead users. This confirmed that the search feature was indeed interacting with the database and was likely vulnerable to SQL injection. Therefore, a more systematic approach was needed to extract useful information from the database.



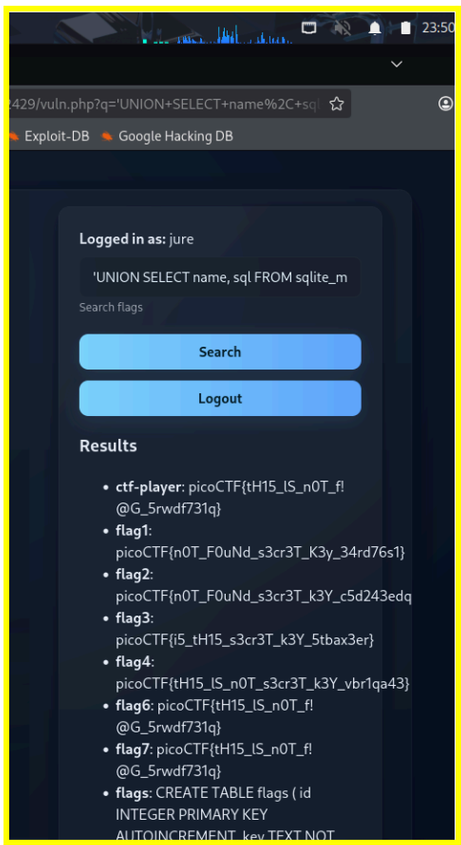
The next step was to determine the number of columns in the database query used by the search function. This was done using the `ORDER BY` SQL injection technique. The query was tested with `ORDER BY 1`, which worked successfully, indicating that at least one column existed. The test was then repeated with `ORDER BY 2`, which also worked. However, when testing `ORDER BY 3`, the application returned an error message indicating that the column index was out of range. From this result, it was determined that the query returned two columns.

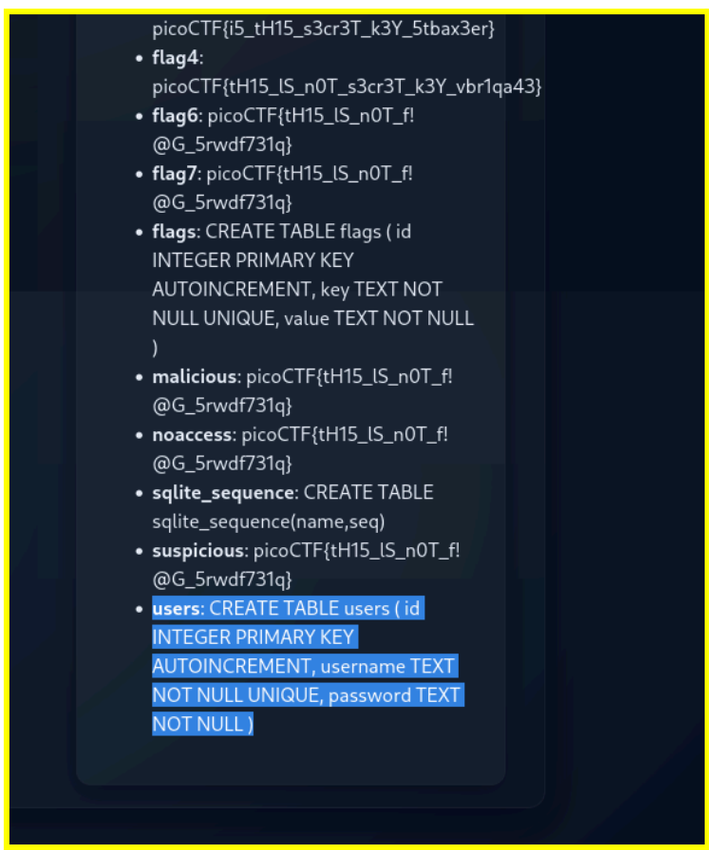




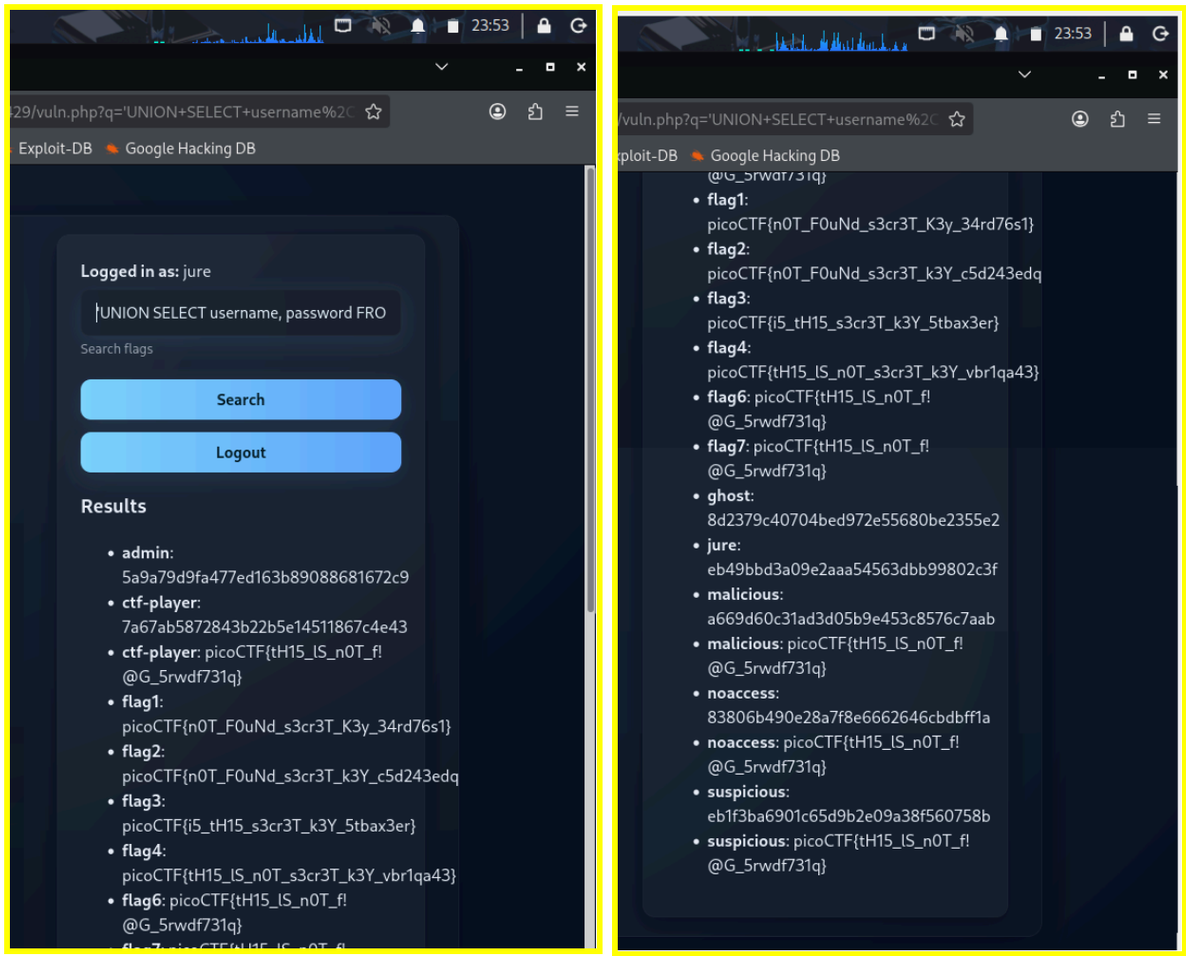


While performing this step, the error message also revealed that the system was using an SQLite database. This information was useful because different database systems have different structures for storing metadata. In SQLite, the database schema information is stored in a table called `sqlite_master`, which can be queried to discover available tables. With this knowledge, a **UNION-based SQL injection** was performed to retrieve the list of tables in the database. The injected query selected the table names and their definitions from `sqlite_master` where the type was `table`. After executing the query through the search field, the results displayed the database structure, revealing tables such as **flags** and **users**.

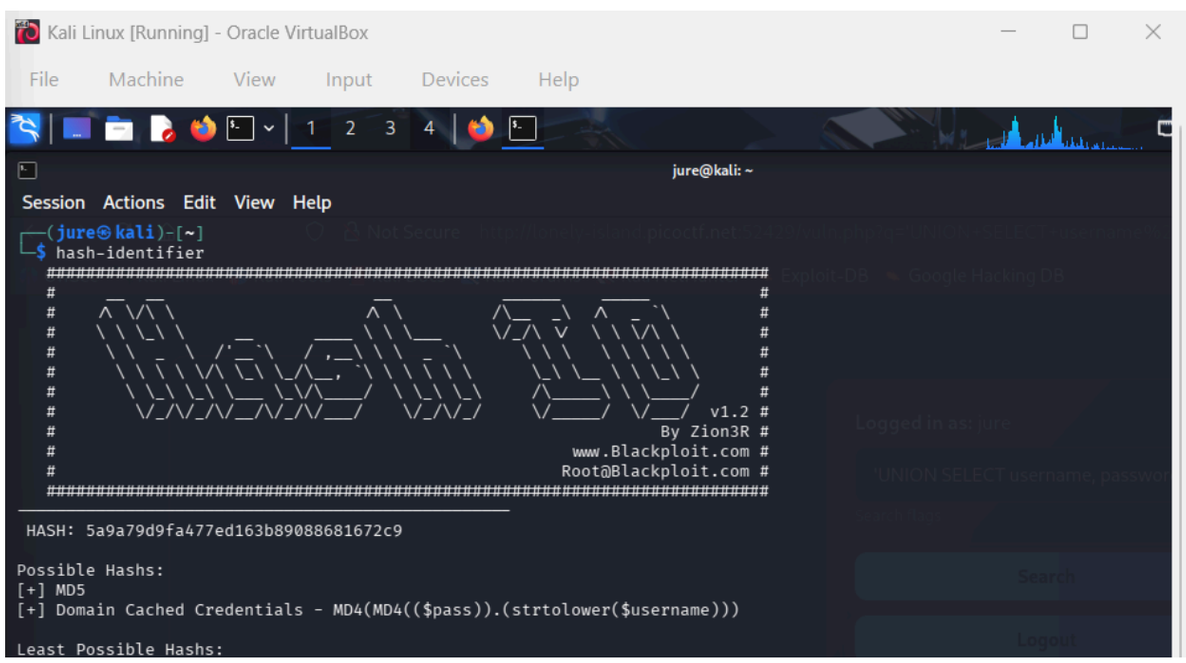




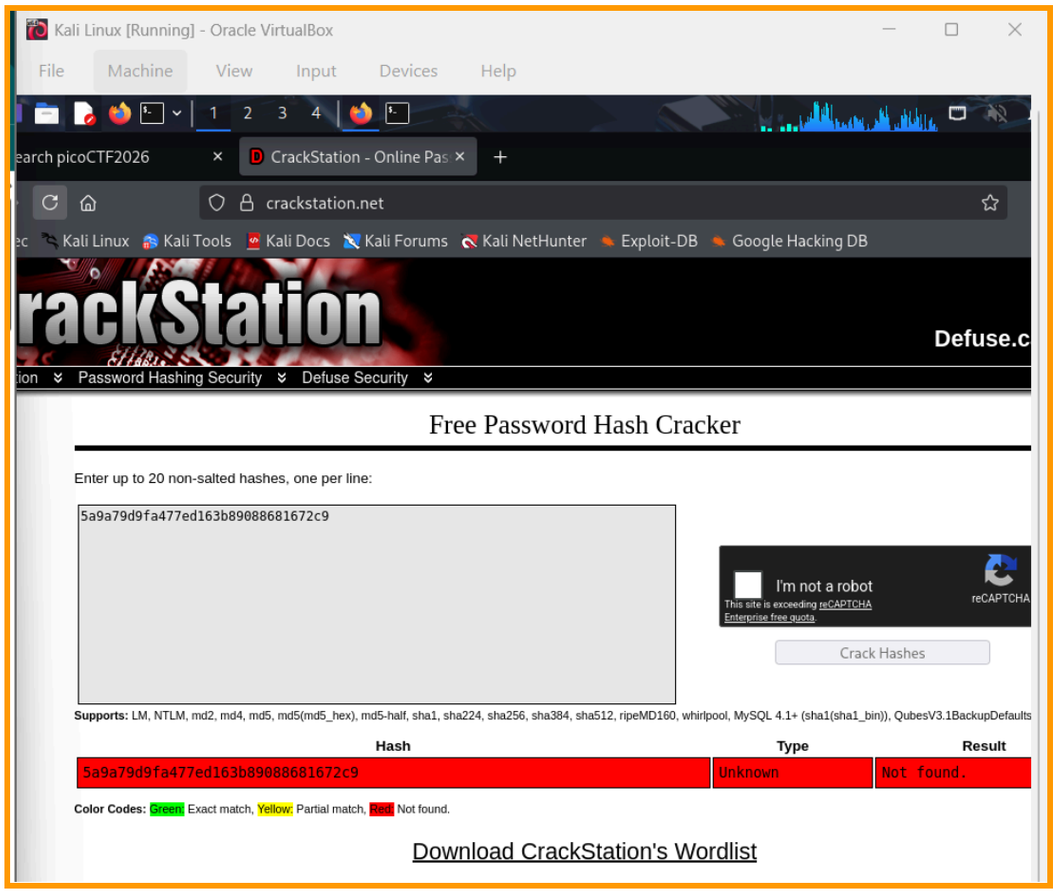
Among these tables, the **users table** was the most relevant because it contained authentication information such as usernames and passwords. To retrieve this information, another UNION-based SQL injection was used to extract the username and password columns from the users table. When the query was executed, the results displayed several usernames along with their corresponding password hashes.



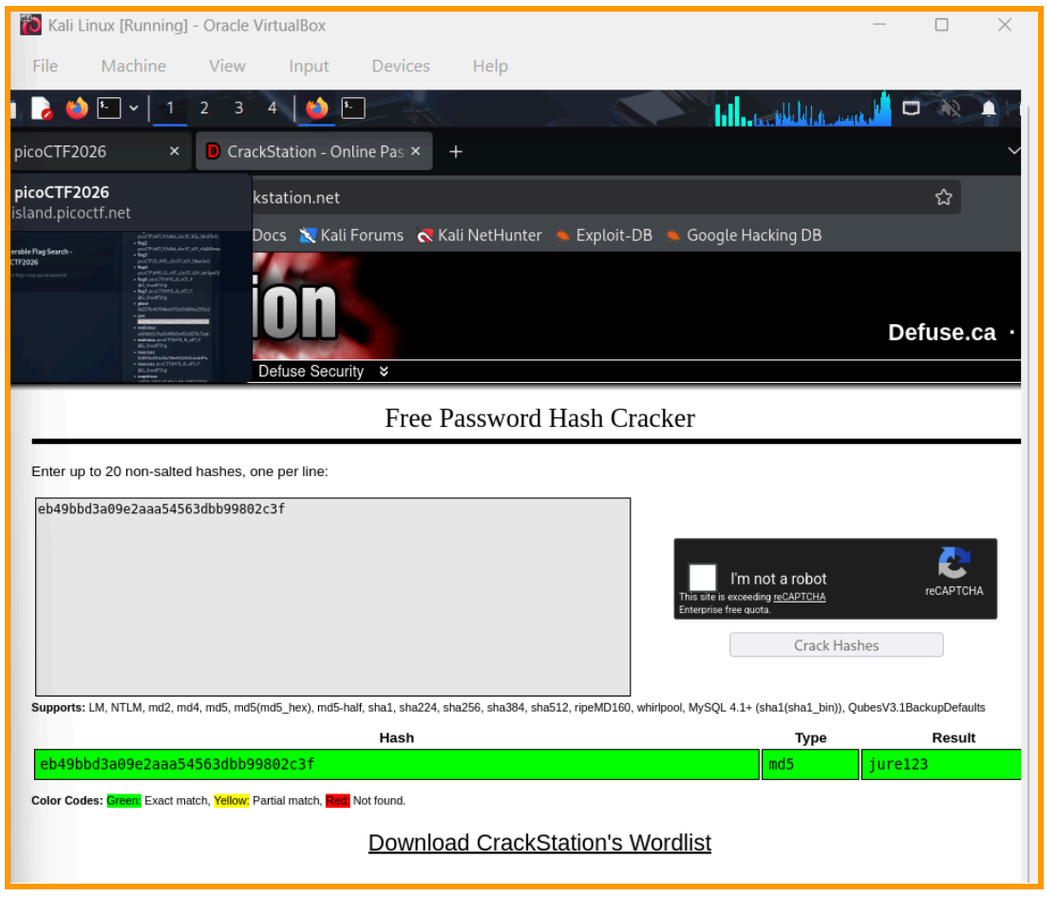
The extracted password credentials were run through a hash-identifier in the Kali Linux terminal, which verified that they were stored as MD5 hashes. This finding was consistent with the hints provided in the initial challenge documentation. Since MD5 is considered a weak hashing algorithm, the hashes were copied and tested using the online hash-cracking tool **CrackStation**. Some hashes were not immediately cracked, but eventually the hash corresponding to the **ctf-player account** was successfully cracked, revealing the password **"dyesebel".**

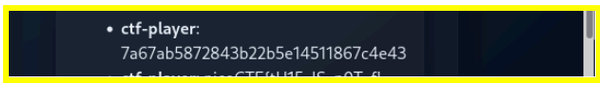


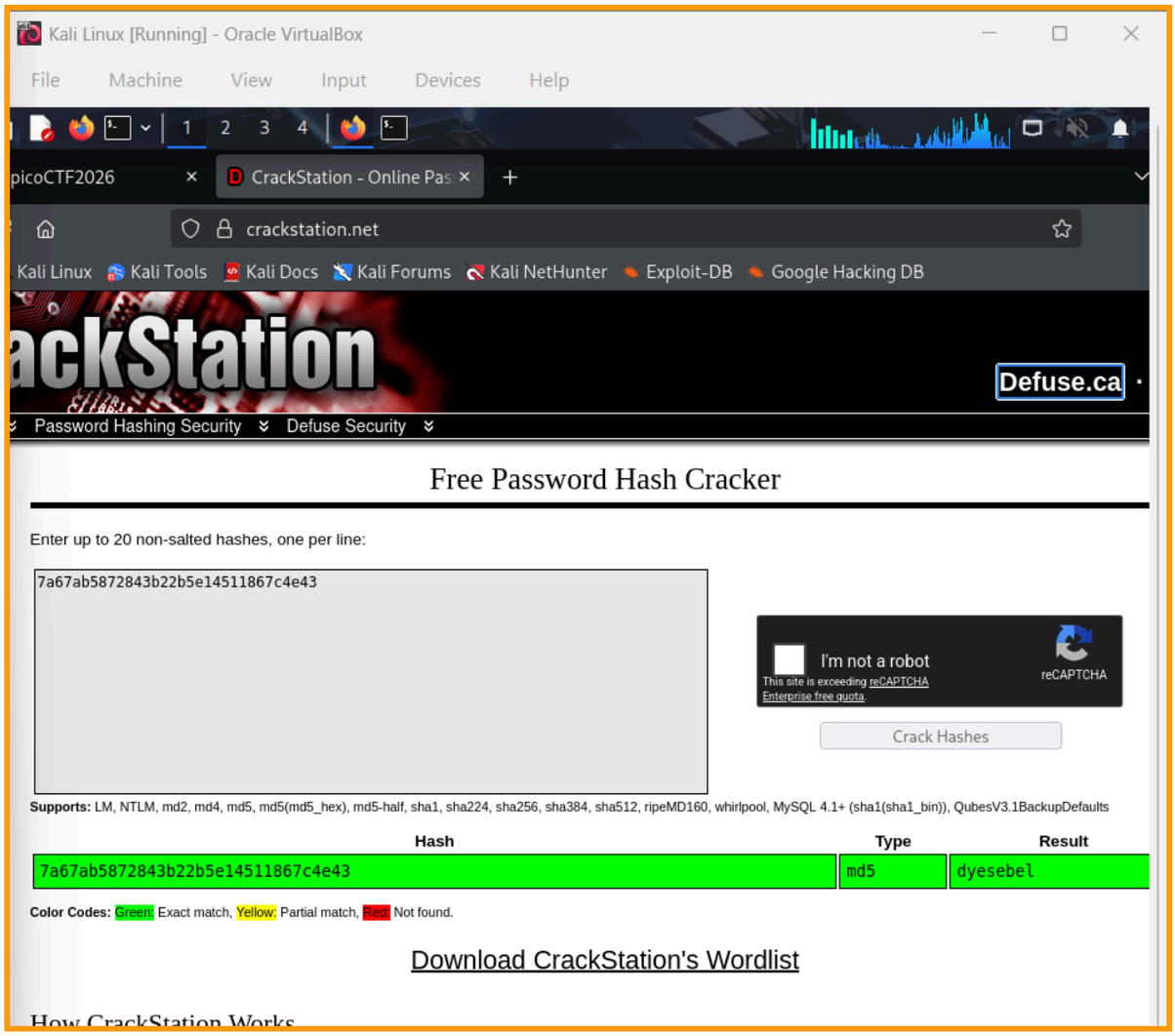
- **admin**:
  5a9a79d9fa477ed163b89088681672c9



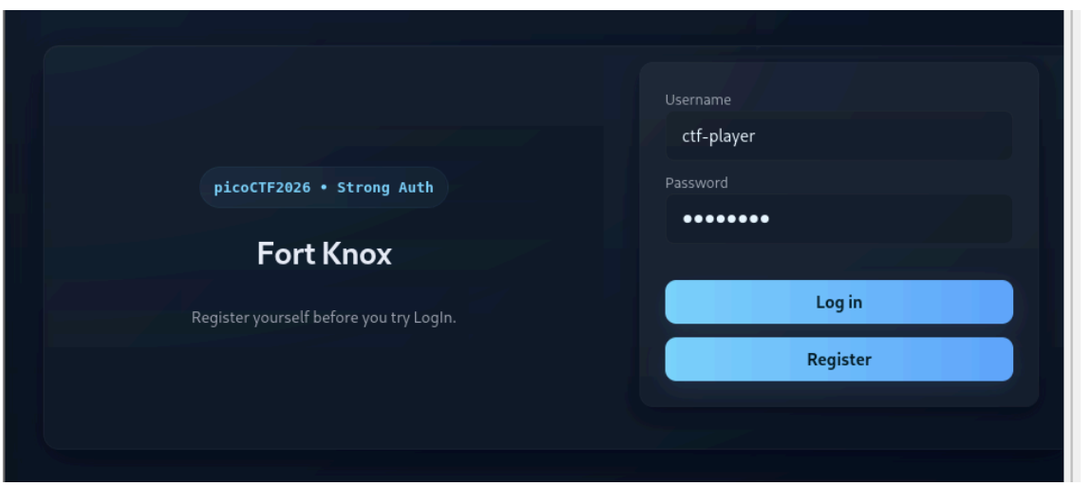
- **jure**:
  eb49bbd3a09e2aaa54563dbb99802c3f

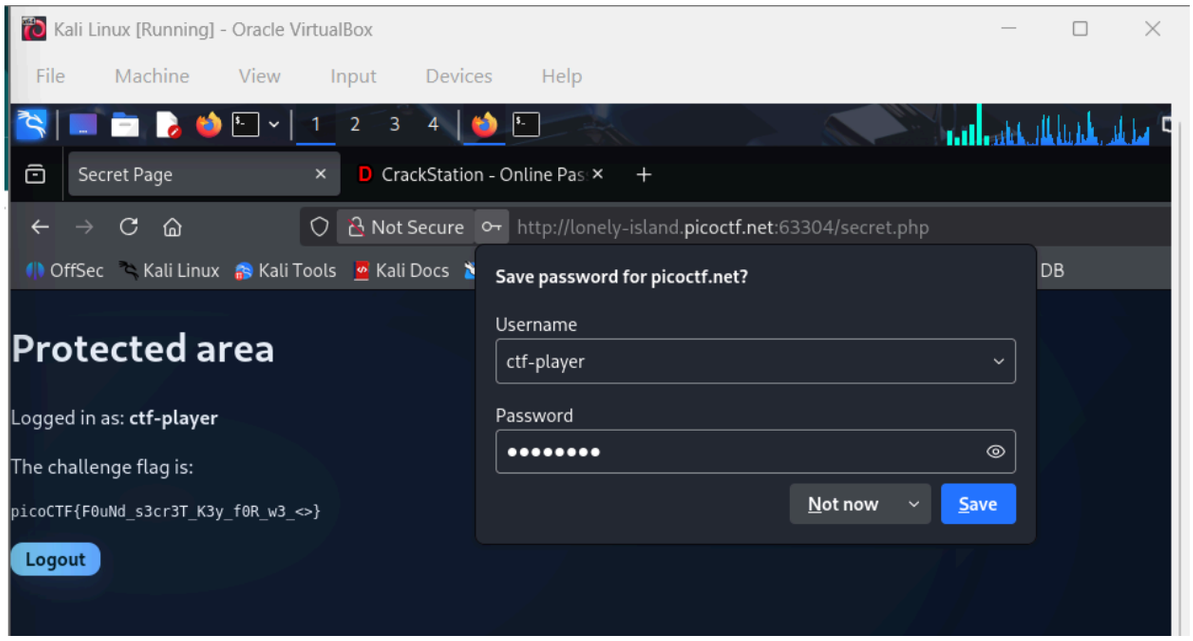


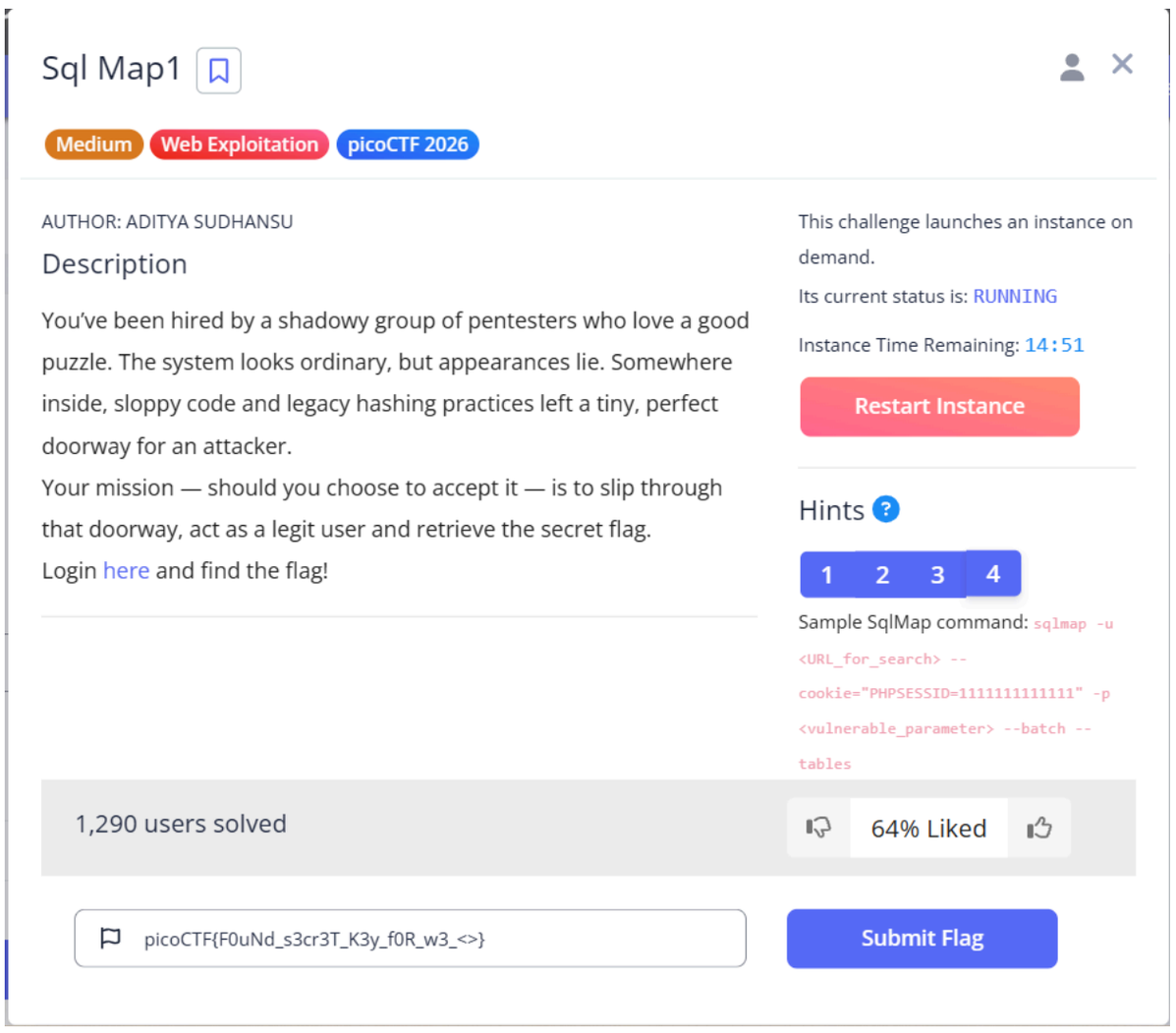


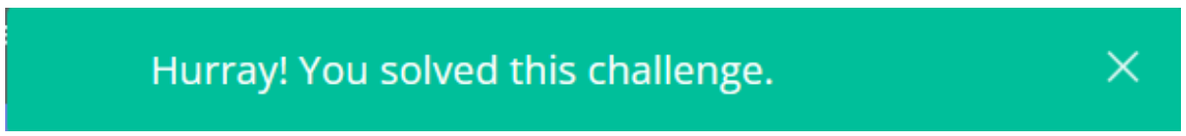


Using these credentials, I logged out of the current account and logged in again using the **ctf-player** username and the cracked password. After successfully logging in as this user, the system displayed the flag, completing the challenge.




Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Secret Page
CrackStation - Online Pas
Not Secure http://lonely-island.picoctf.net:63304/secret.php
OffSec Kali Linux Kali Tools Kali Docs
DB
Protected area
Logged in as: ctf-player
The challenge flag is:
picoCTF{F0uNd_s3cr3T_K3y_f0R_w3_<>}
Logout
Save password for picoctf.net?
Username
ctf-player
Password
Not now
Save


Sql Map1
Medium
Web Exploitation
picoCTF 2026
AUTHOR: ADITYA SUDHANSU
Description
You've been hired by a shadowy group of pentesters who love a good puzzle. The system looks ordinary, but appearances lie. Somewhere inside, sloppy code and legacy hashing practices left a tiny, perfect doorway for an attacker.
Your mission — should you choose to accept it — is to slip through that doorway, act as a legit user and retrieve the secret flag.
Login here and find the flag!
This challenge launches an instance on demand.
Its current status is: RUNNING
Instance Time Remaining: 14:51
Restart Instance
Hints
1 2 3 4
Sample SqlMap command: sqlmap -u <URL_for_search> --cookie="PHPSESSID=1111111111111" -p <vulnerable_parameter> --batch --tables
1,290 users solved
64% Liked
picoCTF{F0uNd_s3cr3T_K3y_f0R_w3_<>}
Submit Flag


Hurray! You solved this challenge.

Overall, the solution involved identifying a SQL injection vulnerability in the search feature, determining the number of columns in the query, extracting the database schema from the SQLite master table, retrieving user credentials, and finally cracking the MD5 hash to gain access to an account that revealed the flag.

Reflection:

This challenge helped me understand how SQL injection vulnerabilities can allow attackers to extract sensitive information from a database when user input is not properly sanitized. By identifying the number of columns, enumerating database tables, and retrieving stored password hashes, I was able to see how small weaknesses in database queries can lead to full data exposure. Overall, the activity highlighted the importance of secure query handling, proper input validation, and stronger password hashing methods to protect web applications.